

ชื่อโครงการ : ระบบอ่านค่ามิเตอร์อุตสาหกรรมและตัวต้านทานด้วยปัญญาประดิษฐ์ (รหัสกลุ่ม MCE04)

รายชื่อสมาชิก : นายชยุต นิธิพลากรณ์ รหัสนักศึกษา [REDACTED]

นายภคินทร เฉลิมชัย รหัสนักศึกษา [REDACTED]

ที่มาและความสำคัญ

ในงานด้านอิเล็กทรอนิกส์และอุตสาหกรรม ผู้ปฏิบัติงานมักต้องอ่านค่าจากอุปกรณ์ต่าง ๆ เช่น แอมป์มิตเตอร์ ตัวต้านทาน และหน้าปัดเกจนิวเมติกหรือไฮดรอลิก ซึ่งอาจเกิดความผิดพลาดจากสายตาได้ โดยเฉพาะเมื่อตัวต้านทานมีแถบสีขนาดเล็ก สีใกล้เคียงกัน หรือเกจมีสเกลละเอียดและต้องเฝ้าระวังอย่างต่อเนื่อง

โครงการนี้จึงพัฒนาระบบช่วยคำนวณค่าตัวต้านทานและเฝ้าระวังเกจอุตสาหกรรม โดยใช้ ESP32S3-CAM เป็นตัวควบคุมหลัก ในส่วนของตัวต้านทาน ผู้ใช้จะป้อนสีของแถบตัวต้านทานผ่านแอป Telegram จากนั้น ESP32-S3 จะคำนวณค่าความต้านทานและส่งผลลัพธ์กลับไปยัง Telegram พร้อมแสดงผลบนจอ OLED I2C

ส่วนการอ่านค่าเกจ ระบบจะใช้กล้อง OV5640 Auto Focus Camera ถ่ายภาพหน้าปัดเกจ แล้วส่งภาพไปให้ Gemini Cloud AI ประมวลผลผ่าน Wi-Fi และ HTTPS เพื่ออ่านค่าที่เพิ่มขึ้นและตรวจสอบสถานะตามแถบสี ได้แก่ สีเขียว สีเหลือง และสีแดง หากอยู่ในแถบสีเขียว ระบบจะแจ้งว่า Safe หากอยู่ในแถบสีเหลืองจะแจ้งให้ระวัง และหากอยู่ในแถบสีแดงจะแจ้งเตือนว่าค่าเกจสูงเกินไป พร้อมส่งเสียงเตือนผ่านลำโพง

โครงการนี้เป็นการประยุกต์ใช้ Embedded System, IoT, Cloud AI, Telegram Bot API, I2C และ I2S เพื่อสร้างต้นแบบระบบช่วยคำนวณและเฝ้าระวังอุปกรณ์อุตสาหกรรม ลดความผิดพลาดจากการอ่านค่าด้วยสายตา และเพิ่มความเร็วในการแจ้งเตือนเมื่อเกิดความผิดปกติ

ภาพรวมระบบ

ระบบนี้เป็นอุปกรณ์ช่วยคำนวณค่าตัวต้านทานและเฝ้าระวังเกจอุตสาหกรรม โดยใช้ ESP32S3-CAM เป็นไมโครคอนโทรลเลอร์หลักของระบบ และใช้กล้อง OV5640 Auto Focus Camera สำหรับถ่ายภาพหน้าปัดเกจ ผู้ใช้สามารถสั่งงานและรับผลลัพธ์ผ่านแอปพลิเคชัน Telegram

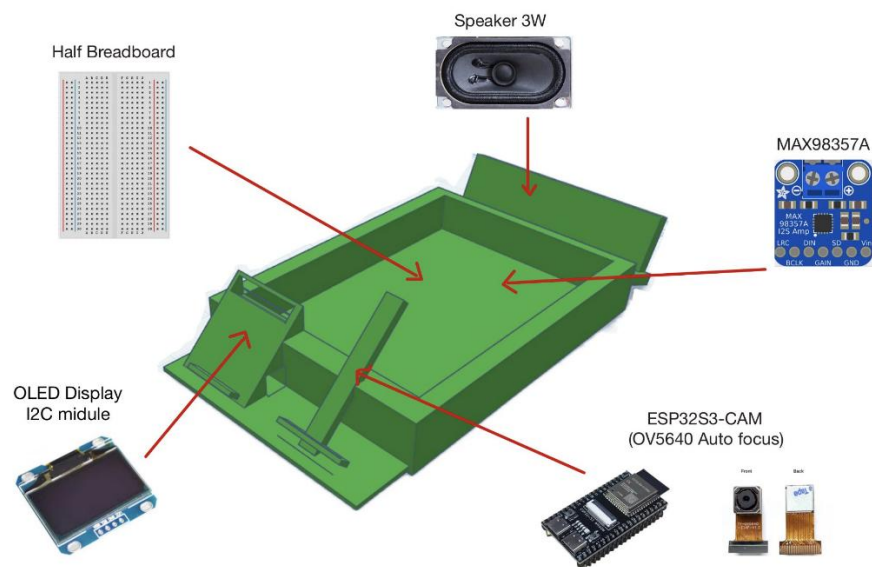
การทำงานของระบบแบ่งออกเป็น 2 โหมดหลัก ได้แก่ 1.) โหมดคำนวณค่าตัวต้านทาน ผู้ใช้ป้อนสีของแถบตัวต้านทานผ่าน Telegram Bot เช่น ตัวต้านทานแบบ 4 แถบ หรือ 5 แถบ จากนั้น ESP32-S3 จะนำข้อมูลที่ได้รับ

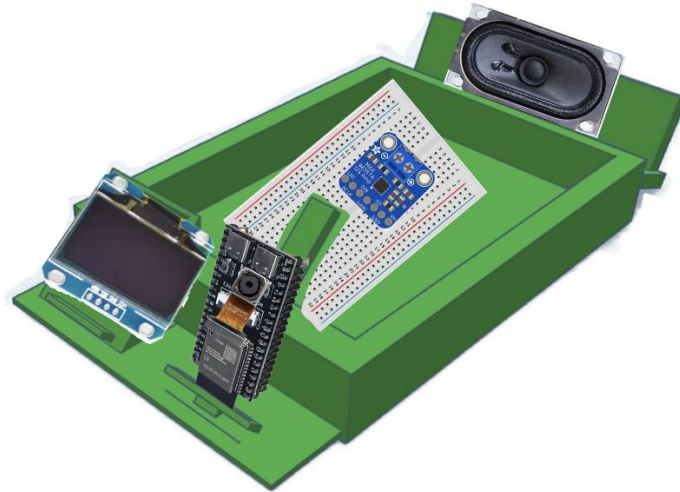
มาคำนวณค่าความต้านทาน ตัวคูณ และค่าความคลาดเคลื่อน แล้วส่งผลลัพธ์กลับไปยัง Telegram พร้อมแสดงผลบนจอ OLED I2C เพื่อให้ผู้ใช้สามารถดูค่าที่คำนวณได้ทันที 2.) โหมดเฝ้าระวังเหตุอุตสาหกรรม ระบบจะใช้กล้อง OV5640 ถ่ายภาพหน้าปัดเกจ เช่น เกจนิวเมติกหรือเกจแรงดันไฮดรอลิก จากนั้น ESP32S3-CAM จะส่งภาพไปยัง Gemini Cloud AI ผ่าน Wi-Fi และ HTTPS API เพื่อวิเคราะห์ตำแหน่งเข็มบนหน้าปัด ค่าที่เกจอ่านได้ และสถานะของแถบสีบนเกจ ได้แก่ สีเขียว สีเหลือง และสีแดง

เมื่อ Gemini วิเคราะห์ผลเสร็จแล้ว ระบบจะส่งผลลัพธ์กลับมายัง ESP32S3-CAM ในรูปแบบข้อมูล เช่น ค่าเกจ หน่วย และสถานะของเกจ จากนั้น ESP32-S3 จะแสดงผลบนจอ OLED I2C และส่งข้อความแจ้งเตือนผ่าน Telegram โดยแบ่งสถานะดังนี้

- แถบสีเขียว: แจ้งว่า Safe หรืออยู่ในสถานะปลอดภัย
- แถบสีเหลือง: แจ้งว่า Warning หรือควรระวัง
- แถบสีแดง: แจ้งว่า Danger หรือค่าที่เกจวัดได้สูงเกินไป พร้อมส่งเสียงเตือนผ่านลำโพง

ระบบนี้จึงสามารถช่วยให้ผู้ใช้ทราบค่าตัวต้านทานและสถานะของเกจได้สะดวกขึ้น ลดความผิดพลาดจากการอ่านค่าด้วยสายตา และเพิ่มความรวดเร็วในการแจ้งเตือนเมื่อเกิดความผิดปกติ



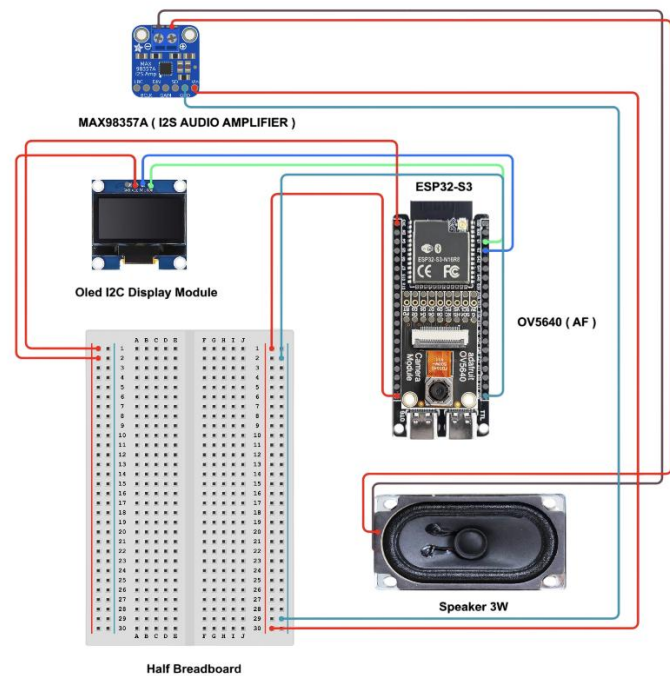


รายการอุปกรณ์

1. OV5640 Auto Focus Camera	1 pc	฿239
2. บอร์ด ESP32S3-CAM OV5640 Camera, Board	1 pc	฿257
3. โมดูลขยายเสียง MAX98357A I2S Audio Amplifier	1 pc	฿70
4. ลำโพง Speaker 3W	1 pc	฿65
5. จอ OLED I2C ขนาด 0.96 นิ้ว	1 pc	฿89
6. สายจัมเปอร์ (ผู้-ผู้+ผู้-เมีย+เมีย-เมีย) แบบแพ็ค	3 pc	฿57
7. Half แบตบอร์ด	1 pc	฿24
8. สาย USB type-A	1 pc	฿18
9. ค่า print เคสชิ้นงาน	1 pc	฿100

รวมงบประมาณ 951 บาท

แผนผังการเชื่อมต่ออุปกรณ์และโปรโตคอลการสื่อสาร



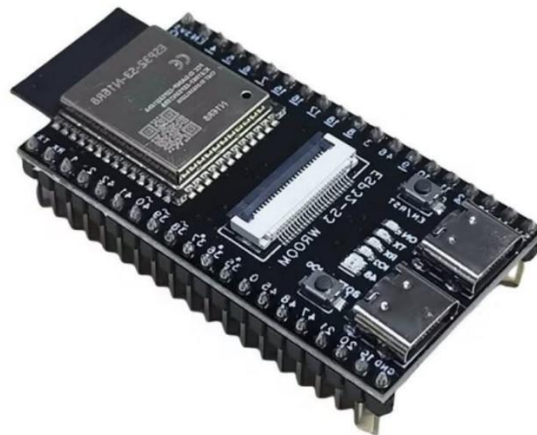
<u>External device</u>	<u>External device pin</u>	<u>Microcontroller pin</u>	<u>Protocol</u>
OV5640 AF Camera	FPC Camera connector	ESP32-S3-CAM camera connector	DVP / SCCB
OLED SSD1306 0.96 นิ้ว	SDA, SDL	GPIO1, GPIO2	I2C
Gemini AI API, Telegram Bot API	Wi-Fi	ESP32-S3 Wi-Fi	HTTPS / JSON
Mobile Phone (Web Browser)	Wi-Fi	ESP32-S3 Wi-Fi	HTTP
MAX98357A Speaker	DIN, BCLK, LRC	GPIO10, GPIO11, GPIO12	I2S
Gauge Monitoring System	JPEG Image	ESP32-S3 Wi-Fi	HTTPS / Base64 / JSON

9. การเลือกไมโครคอนโทรลเลอร์

สเปกและความสามารถที่เกี่ยวข้องกับโครงการ

ESP32S3-CAM เป็นบอร์ดไมโครคอนโทรลเลอร์ที่เหมาะสมกับโครงการนี้ เนื่องจากมี Wi-Fi ในตัว รองรับ การเชื่อมต่อกล้อง OV5640 Auto Focus Camera ที่สำคัญคือมี PRAM ทำให้สามารถส่งภาพได้ และสามารถ เชื่อมต่อกับอุปกรณ์ภายนอกได้หลายรูปแบบ เช่น จอ OLED I2C และโมดูลขยายเสียง MAX98357A ผ่าน I2S

ในโครงการนี้ ESP32S3-CAM ทำหน้าที่เป็นตัวควบคุมหลักของระบบ โดยรับข้อมูลสีตัวต้านทานจากผู้ใช้ ผ่าน Telegram Bot แล้วนำมาคำนวณค่าความต้านทานภายในบอร์ด จากนั้นส่งผลลัพธ์กลับไปยัง Telegram และแสดงผลบนจอ OLED I2C ส่วนในโหมดอ่านค่าเกจ ระบบจะใช้กล้อง OV5640 ถ่ายภาพหน้าปัดเกจ แล้วส่ง ภาพไปยัง Gemini Cloud AI เพื่อวิเคราะห์ค่าที่เพิ่มขึ้นและสถานะของเกจ



ภาพตัวอย่างบอร์ด ESP32S3-CAM

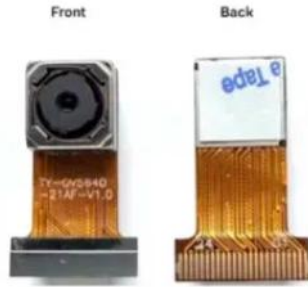
Feature สำคัญที่เลือกใช้ในโครงการ

1.) Wi-Fi Communication

ใช้สำหรับเชื่อมต่ออินเทอร์เน็ต เพื่อรับคำสั่งจาก Telegram Bot ส่งผลลัพธ์กลับไปยังผู้ใช้ และส่งภาพเกจไปยัง Gemini Cloud AI

2.) Camera Interface

ใช้เชื่อมต่อกับกล้อง OV5640 Auto Focus Camera สำหรับถ่ายภาพหน้าปัดเกจที่ต้องการวิเคราะห์



ภาพตัวอย่างกล้อง OV5640 AF



ภาพตัวอย่างจอ OLED I2C MODULE SSD1306

3.) I2C Interface

ใช้เชื่อมต่อกับจอ OLED I2C เพื่อแสดงค่าตัวต้านทาน ค่าเกจ และสถานะการทำงานของระบบ

4.) I2S Interface

ใช้เชื่อมต่อกับโมดูล MAX98357A สำหรับขับลำโพง เพื่อแจ้งเตือนด้วยเสียงเมื่อค่าเกจอยู่ในระดับอันตราย



ภาพตัวอย่างโมดูล MAX98357A



ภาพตัวอย่าง Speaker 3W

5.) GPIO Control

ใช้ควบคุมอุปกรณ์ภายนอก เช่น จอ OLED โมดูลเสียง และอุปกรณ์เสริมอื่นๆ ที่เกี่ยวข้อง

6.) ขนาดเล็กและเหมาะกับการทำต้นแบบ

ตัวบอร์ดมีขนาดกะทัดรัด สามารถติดตั้งร่วมกับกล้อง จอ OLED ลำโพง และแหล่งจ่ายไฟในเคสต้นแบบได้ง่าย

10. การสื่อสารผ่าน HTTPS API ระหว่าง ESP32S3-CAM กับ Gemini Cloud AI และ Telegram Bot API

10.1 สเปกอุปกรณ์ที่เกี่ยวข้อง

อุปกรณ์หลักที่เกี่ยวข้องคือ ESP32S3-CAM OV5640 Camera Board ทำหน้าที่เป็นไมโครคอนโทรลเลอร์หลักของระบบ มี Wi-Fi ในตัว จึงสามารถเชื่อมต่ออินเทอร์เน็ตและติดต่อกับบริการภายนอก เช่น Gemini Cloud AI และ Telegram Bot API ได้

กล้อง OV5640 Auto Focus Camera ใช้สำหรับถ่ายภาพหน้าปัดเกจ เช่น เกจนิวเมติกหรือเกจไฮดรอลิก เพื่อส่งภาพไปให้ Gemini Cloud AI วิเคราะห์ค่าที่เพิ่มขึ้นและสถานะของเกจ ส่วนการคำนวณค่าตัวต้านทานจะไม่ใช้กล้องแล้ว แต่ให้ผู้ใช้ป้อนสีของแถบตัวต้านทานผ่าน Telegram Bot จากนั้น ESP32S3-CAM จะคำนวณค่าความต้านทานเอง

ระบบแสดงผลผ่านจอ OLED I2C และแจ้งเตือนด้วยเสียงผ่านโมดูล MAX98357A กับลำโพง โดย Telegram Bot API ใช้สำหรับรับคำสั่งจากผู้ใช้ ส่งผลการคำนวณตัวต้านทาน และส่งข้อความแจ้งเตือนสถานะของเกจและค่าความดัน ส่วน **Telegram Bot API** ใช้สำหรับส่งข้อความแจ้งเตือนเมื่อค่าความดันเกิน Limit ที่กำหนดไว้

10.2 Protocol ที่ใช้สื่อสาร

Protocol หลักที่เลือกใช้คือ HTTPS API โดย ESP32S3-CAM จะเชื่อมต่อ Wi-Fi เพื่อสื่อสารกับ Telegram Bot API และ Gemini Cloud AI ผ่านอินเทอร์เน็ต ซึ่ง HTTPS มีการเข้ารหัสข้อมูล ทำให้ปลอดภัยกว่า HTTP ทั่วไป

ในโหมดคำนวณค่าตัวต้านทาน ผู้ใช้จะส่งสีของแถบตัวต้านทานผ่าน Telegram Bot เช่น น้ำตาล ดำ แดง ทอง จากนั้น ESP32S3-CAM จะรับข้อมูลสี แปลงเป็นค่าตัวเลข ตัวคูณ และค่าความคลาดเคลื่อน แล้วคำนวณค่าความต้านทาน ผลลัพธ์ที่ได้จะถูกส่งกลับไปยัง Telegram และแสดงบนจอ OLED I2C

ในโหมดอ่านค่าเกจ กล้อง OV5640 จะถ่ายภาพหน้าปัดเกจ จากนั้น ESP32S3-CAM จะแปลงภาพเป็นข้อมูลที่เหมาะสม เช่น Base64 แล้วส่งไปยัง Gemini Cloud AI ผ่าน HTTPS API เพื่อวิเคราะห์ค่าที่เพิ่มขึ้นและ

แถบสีของगेज เมื่อ Gemini วิเคราะห์เสร็จ จะส่งผลลัพธ์กลับมายัง ESP32S3-CAM ในรูปแบบ JSON เช่น ค่าगेज หน่วย และสถานะของगेज

ระบบจะแบ่งสถานะของगेजเป็น 3 ระดับ ได้แก่ สีเขียวคือ Safe เหลืองคือ Warning หรือควรระวัง และแดงคือ Danger หรือค่าที่गेजวัดได้สูงเกินไป หากอยู่ในสถานะสีแดง ระบบจะส่งข้อความแจ้งเตือนผ่าน Telegram และสั่งให้ลำโพงส่งเสียงเตือนทันที ข้อมูลแบบ JSON และ Base64 ไม่ใช่ Protocol โดยตรง แต่เป็นรูปแบบข้อมูลที่ใช้ร่วมกับ API เพื่อให้ ESP32S3-CAM สามารถรับส่งข้อมูลกับ Telegram และ Gemini Cloud AI ได้สะดวกขึ้น

10.3 Library และตัวอย่าง Code ที่เกี่ยวข้องกับ HTTPS

Library ที่ใช้ใน HTTPS ได้แก่ WiFi.h, HTTPClient.h, WiFiClientSecure.h, ArduinoJson.h, base64.h

ตัวอย่าง Code ที่เกี่ยวข้อง

```
WiFiClientSecure client;

client.setInsecure();

HTTPClient https;

String url =

"https://generativelanguage.googleapis.com/v1beta/models/gemini-2.5-

flash:generateContent?key="

+ GEMINI_API_KEY;

https.begin(client, url);

https.addHeader("Content-Type", "application/json");

int code = https.POST(payload);

String response = https.getString();

https.end();
```

11. ผลงานที่ทำสำเร็จ และผลทดสอบการทำงาน

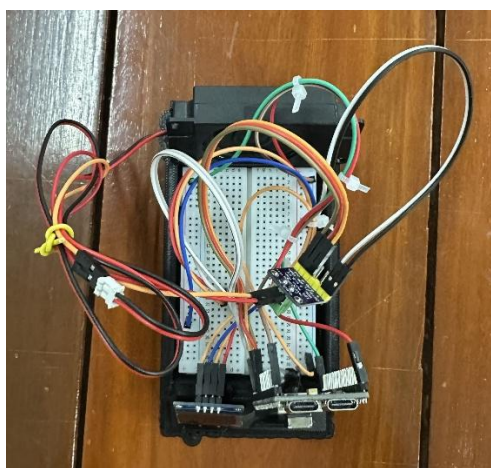
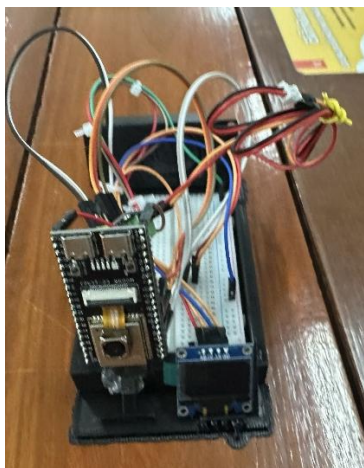
จากการพัฒนาโครงงาน ระบบสามารถทำงานได้ตามวัตถุประสงค์หลัก คือ สามารถคำนวณค่าตัวต้านทาน แสดงผลผ่านจอ OLED I2C และสามารถอ่านค่าจากเกจโดยใช้กล้องร่วมกับ Gemini Cloud AI เพื่อวิเคราะห์ค่า และสถานะของเกจได้ โดยผลลัพธ์สามารถแสดงได้ทั้งบนจอ OLED และส่งแจ้งเตือนผ่าน Telegram

11.1 ผลงานที่ทำสำเร็จ

ระบบต้นแบบที่พัฒนาขึ้นประกอบด้วย ESP32S3-CAM, กล้อง OV5640 Auto Focus Camera, จอ OLED I2C, โมดูลขยายเสียง MAX98357A และลำโพง โดยสามารถแบ่งการทำงานออกเป็น 2 โหมดหลัก ได้แก่ โหมดคำนวณค่าตัวต้านทาน และโหมดอ่านค่าเกจ

ในโหมดคำนวณค่าตัวต้านทาน ระบบสามารถรับค่าสีของแถบตัวต้านทานจากผู้ใช้ แล้วให้ ESP32-S3 คำนวณค่าความต้านทานภายในบอร์ดโดยไม่ต้องส่งไปประมวลผลผ่าน Cloud AI จากนั้นระบบสามารถแสดงผลลัพธ์บนจอ OLED I2C และบนหน้า Interface ได้ เช่น ผลการทดสอบแสดงค่า 200.00 ohm $\pm 1\%$ พร้อมข้อความว่าเป็นการคำนวณแบบ Offline โดย ESP32-S3

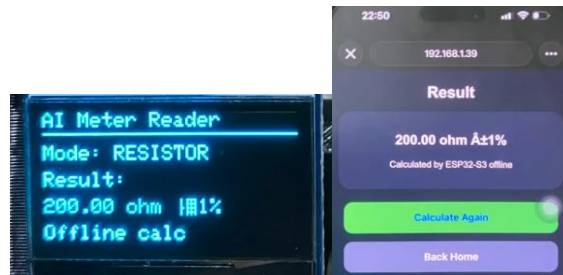
ในโหมดอ่านค่าเกจ ระบบสามารถใช้กล้อง OV5640 ถ่ายภาพหน้าปัดเกจ แล้วส่งภาพไปให้ Gemini Cloud AI วิเคราะห์ค่าที่เข็มชี้และสถานะของเกจ จากนั้นระบบจะแสดงผลลัพธ์บนจอ OLED I2C และส่งข้อความแจ้งเตือนผ่าน Telegram โดยแบ่งสถานะเป็น 3 ระดับ ได้แก่ สีเขียวคือ Safe สีเหลืองคือ Warning และสีแดงคือ Danger



ภาพผลงานที่สำเร็จ

11.2 ผลการทดสอบโหมดคำนวณค่าตัวต้านทาน

จากการทดสอบโหมดคำนวณค่าตัวต้านทาน พบว่าระบบสามารถคำนวณค่าความต้านทานได้ถูกต้องตาม
สีที่ผู้ใช้งานกำหนด โดย ESP32-S3 สามารถประมวลผลภายในบอร์ดได้โดยไม่ต้องใช้ Gemini Cloud AI

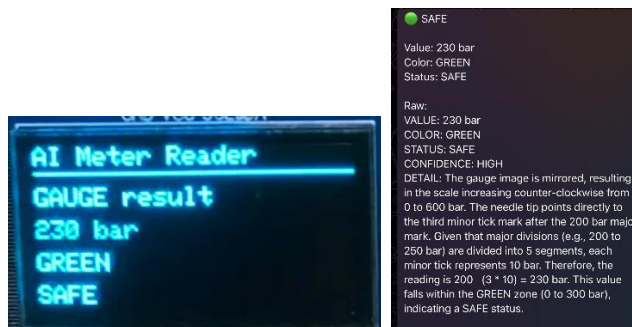


ภาพตัวอย่างการอ่านค่าตัวต้านทาน

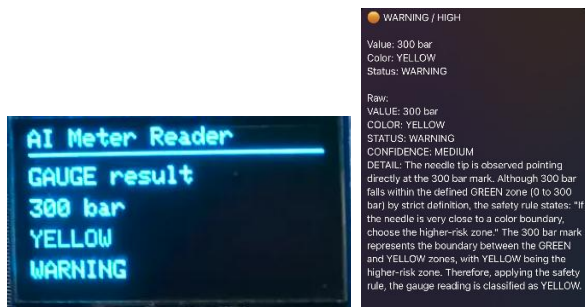
ผลการทดสอบแสดงค่าความต้านทาน $200.00\text{ ohm} \pm 1\%$ บนจอ OLED I2C และแสดงผลเดียวกันบน
หน้า Interface ของผู้ใช้ แสดงให้เห็นว่าระบบสามารถคำนวณค่าตัวต้านทานและส่งผลลัพธ์ไปยังส่วนแสดงผลได้
ถูกต้อง และข้อดีของการคำนวณแบบ Offline คือช่วยลดการใช้งาน API ลดความล่าช้าในการประมวลผล และลด
ความผิดพลาดจากการใช้กล้องอ่านแถบสีตัวต้านทาน เนื่องจากแถบสีตัวต้านทานมีขนาดเล็กและอาจอ่านผิดได้
ง่ายหากใช้กล้อง

11.3 ผลการทดสอบโหมดอ่านค่าเกจ

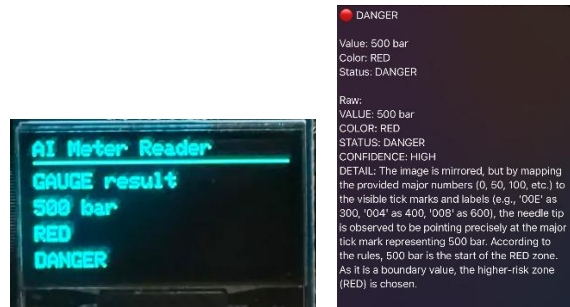
จากการทดสอบโหมดอ่านค่าเกจ ระบบสามารถส่งภาพจากกล้อง OV5640 ไปยัง Gemini Cloud AI
เพื่อวิเคราะห์ค่าที่เข็มชี้และสถานะของเกจได้ โดยผลลัพธ์ที่ได้สามารถนำมาแสดงบนจอ OLED I2C และส่งแจ้ง
เตือนผ่าน Telegram ได้สำเร็จ



ภาพตัวอย่างการอ่านค่าเกจวัดได้ 230 bar



ภาพตัวอย่างอ่านค่าเกจวัดได้ 300 bar



ภาพตัวอย่างอ่านค่าเกจวัดได้ 500 bar

ผลการทดสอบแบ่งได้ดังนี้

ค่าที่ทดสอบ		สีที่ระบบตรวจพบ	สถานะ	ผลการทำงาน
230 bar	GREEN	SAFE	แสดงผลบน OLED และแจ้งสถานะปลอดภัย	
300 bar	YELLOW	WARNING	แสดงผลบน OLED และส่งแจ้งเตือนผ่าน Telegram	
500 bar	RED	DANGER	แสดงผลบน OLED และส่งแจ้งเตือนอันตรายผ่าน Telegram	

จากผลการทดสอบที่ค่า **230 bar** ระบบสามารถวิเคราะห์ว่าเกจอยู่ในแถบสีเขียว และแสดงสถานะ **SAFE** บนจอ OLED ได้ถูกต้อง ผลการทดสอบที่ค่า **300 bar** ระบบสามารถวิเคราะห์ว่าเข็มอยู่บริเวณขอบเขตระหว่างสีเขียวและสีเหลือง โดยระบบเลือกสถานะที่มีความเสี่ยงสูงกว่า คือ **YELLOW / WARNING** และส่งข้อความแจ้งเตือนผ่าน Telegram ได้สำเร็จ ผลการทดสอบที่ค่า **500 bar** ระบบสามารถวิเคราะห์ว่าเข็มอยู่ในแถบสีแดง และแสดงสถานะ **RED / DANGER** พร้อมส่งข้อความแจ้งเตือนผ่าน Telegram ได้ถูกต้อง แสดงให้เห็นว่าระบบสามารถตรวจจับสถานะอันตรายของเกจได้ตามเงื่อนไขที่กำหนด

11.4 สรุปผลการทดสอบ

จากการทดสอบทั้งหมด ระบบสามารถทำงานได้ตามเป้าหมายหลักของโครงการ โดยสามารถคำนวณค่าตัวต้านทานด้วย ESP32-S3 แสดงผลผ่านจอ OLED I2C และสามารถอ่านค่าเกจโดยใช้กล้องร่วมกับ Gemini Cloud AI ได้ ระบบสามารถจำแนกสถานะของเกจเป็น Safe, Warning และ Danger พร้อมแสดงผลและแจ้งเตือนผ่าน Telegram ได้สำเร็จ

12. ปัญหาและการแก้ปัญหา

12.1 ปัญหาที่พบบ่อยในการทำงาน และการแก้ปัญหา

1. ปัญหาเว็บ Preview เกจค้างหรือทำงานช้า

สาเหตุ : ระบบส่งภาพต่อเนื่องคล้ายวิดีโอ ทำให้ ESP32-S3 ประมวลผลหนัก

วิธีแก้ไข : ปรับให้ส่งภาพเฉพาะเมื่อผู้ใช้กด Preview หรือ Analyze เท่านั้น ลดภาระการประมวลผลและ ทำให้เว็บตอบสนองเร็วขึ้น

2. ปัญหาภาพไม่ชัด ทำให้วิเคราะห์ค่าไม่แม่นยำ

สาเหตุ : ตั้งค่ากล้องให้เน้นความเร็วมากเกินไป ทำให้รายละเอียดของหน้าปัดเกจไม่ชัด

วิธีแก้ไข : ปรับความละเอียดเป็น VGA และตั้งค่า JPEG Quality เป็น 10 เพื่อเพิ่มความคมชัดของภาพ

3. ปัญหา ESP32-S3 รีบูตเมื่อใช้แหล่งจ่ายไฟภายนอก

สาเหตุ : แรงดันไฟหรือกระแสจากแหล่งจ่ายภายนอกไม่เสถียร ทำให้บอร์ดรีบูตตลอดเวลา

วิธีแก้ไข : เปลี่ยนมาจ่ายไฟผ่านพอร์ต USB-C ของบอร์ด ทำให้ไฟเลี้ยงเสถียรและระบบทำงานได้ปกติ

4. ปัญหากล้องรุ่นเดิมไม่เหมาะกับการใช้งาน

สาเหตุ : กล้อง OV2640 ไม่มี Auto Focus และการส่งภาพมีข้อจำกัด ทำให้ภาพไม่คมชัด

วิธีแก้ไข : เปลี่ยนมาใช้กล้อง OV5640 Auto Focus เพื่อเพิ่มความคมชัดและความสะดวกในการใช้งาน

5. ปัญหาเสียงแจ้งเตือนมีความล่าช้า

สาเหตุ : ใช้ Gemini สร้างเสียงทุกครั้ง ทำให้ต้องรอการประมวลผลผ่านอินเทอร์เน็ต

วิธีแก้ไข : แปลงไฟล์เสียงเป็น WAV และฝังเป็นไฟล์ .h ทำให้สามารถเล่นเสียงได้ทันที

6. ปัญหาไฟล์เสียงเตือนมีขนาดใหญ่เกินไป

สาเหตุ : เสียงเตือนอันตรายมีความยาวมาก ใช้หน่วยความจำสูง

วิธีแก้ไข : เปลี่ยนเป็นเสียง Beep สั้น ๆ วนซ้ำแทน เพื่อลดการใช้หน่วยความจำ

7. ปัญหาการอ่านค่าเกจในโหมด Auto ใช้ภาพเก่า

สาเหตุ : กล้องบางครั้งส่งเฟรมเดิมซ้ำ ทำให้ผลการวิเคราะห์ไม่ตรงกับสถานะปัจจุบัน

วิธีแก้ไข : เคลียร์เฟรมเก่า 3 ครั้งก่อนถ่ายภาพใหม่ เพื่อให้ได้ภาพล่าสุดสำหรับการวิเคราะห์

8. ปัญหาการอ่านค่าตัวเลขบนเกจยังมีความคลาดเคลื่อน

สาเหตุ : การประมาณตำแหน่งเข็มและสเกลจากภาพมีความซับซ้อน

วิธีแก้ไข : ปรับ Prompt ของ Gemini ให้ระบุวิธีอ่านเข็ม สเกล และช่วงสเกลอย่างละเอียด ทำให้ผลการวิเคราะห์แม่นยำขึ้น

12.2 ปัญหาที่เหลืออยู่ และแนวทางการแก้ปัญหา

1. ปัญหาแสงสะท้อนบนหน้าปัดเกจ

สาเหตุ : เมื่อมีแสงสะท้อนบนกระจกหน้าปัด ระบบอาจอ่านตำแหน่งเข็มผิดพลาด

แนวทางแก้ไข : ใช้กล้องที่มีคุณภาพสูงขึ้น หรือปรับตำแหน่งกล้องและแสงสว่างให้เหมาะสมเพื่อลดการสะท้อน

2. ปัญหาข้อจำกัดด้าน Gemini API

สาเหตุ : Gemini API มีข้อจำกัดด้าน Token และจำนวนรูปที่สามารถวิเคราะห์ได้ต่อวัน

แนวทางแก้ไข : พัฒนาระบบด้วย TinyML หรือ Edge AI ให้สามารถประมวลผลบน ESP32-S3 ได้โดยตรง เพื่อลดการพึ่งพา Cloud AI และเพิ่มความรวดเร็วในการทำงาน

13. แนวทางการพัฒนาโครงการ

จากการพัฒนาโครงการระบบคำนวณค่าตัวต้านทานและเฟิร์มแวร์เกจอุตสาหกรรม พบว่ายังมีส่วนที่สามารถพัฒนาต่อยอดให้ระบบใช้งานได้สะดวกและเหมาะสมกับผู้ใช้งานทั่วไปมากขึ้น โดยมีแนวทางการพัฒนาดังนี้

13.1 การพัฒนา Interface สำหรับเลือกสีตัวต้านทาน

ในระบบปัจจุบัน การคำนวณค่าตัวต้านทานใช้วิธีให้ผู้ใช้อัปโหลดหรือเลือกชื่อสีของแถบตัวต้านทานผ่านแอป Telegram เช่น ดำ น้ำตาล แดง ส้ม เหลือง เขียว น้ำเงิน ม่วง เทา ขาว ทอง และเงิน จากนั้น ESP32S3-CAM จะนำข้อมูลสีที่ได้รับไปคำนวณค่าความต้านทาน

อย่างไรก็ตาม การเลือกสีจากชื่อสีเพียงอย่างเดียวอาจทำให้ผู้ใช้สับสนหรือเลือกผิดได้ โดยเฉพาะผู้ที่ยังไม่คุ้นเคยกับการอ่านรหัสสีตัวต้านทาน ดังนั้นแนวทางการพัฒนาต่อไปคือการปรับ Interface ใน Telegram ให้มีลักษณะเป็นปุ่มสี หรือเมนูที่แสดงสีของแถบตัวต้านทานให้เห็นชัดเจน เช่น ปุ่มสีดำ สีน้ำตาล สีแดง สีส้ม สีเหลือง เป็นต้น เพื่อให้ผู้ใช้สามารถเลือกสีได้จากภาพหรือสีของปุ่มโดยตรง แทนการอ่านข้อความเพียงอย่างเดียว

การพัฒนาในส่วนนี้จะช่วยให้การใช้งานง่ายขึ้น ลดความผิดพลาดจากการเลือกชื่อสีผิด และทำให้ระบบเหมาะสมกับผู้ใช้งานทั่วไปมากขึ้น โดยเฉพาะในกรณีที่ผู้ใช้ต้องการคำนวณค่าตัวต้านทานแบบรวดเร็วผ่านโทรศัพท์มือถือ นอกจากนี้ยังสามารถพัฒนาให้รองรับการเลือกตัวต้านทานทั้งแบบ 4 แถบ และ 5 แถบ พร้อมแสดงตัวอย่างสีที่ผู้ใช้เลือกก่อนยืนยันการคำนวณ เพื่อเพิ่มความถูกต้องของผลลัพธ์

13.2 การพัฒนา Interface สำหรับตั้งค่าช่วงสีของเกจ

ในระบบปัจจุบัน การวิเคราะห์สถานะของเกจจะทำการกำหนดช่วงค่าของแต่ละแถบสีไว้ในโปรแกรม เช่น ช่วงสีเขียวเป็นค่าปลอดภัย ช่วงสีเหลืองเป็นค่าควรระวัง และช่วงสีแดงเป็นค่าอันตราย ซึ่งการกำหนดค่าเหล่านี้ต้องแก้ไขภายในโค้ดของ ESP32S3-CAM โดยตรง

ข้อจำกัดของวิธีนี้คือ หากผู้ใช้ต้องการเปลี่ยนไปใช้เกจคนละรุ่น หรือเกจที่มีช่วงการวัดแตกต่างกัน เช่น 0–10 bar, 0–100 psi หรือ 0–1000 kPa ผู้ใช้จะต้องแก้ไขค่าในโค้ดใหม่ทุกครั้ง ซึ่งไม่เหมาะสมกับผู้ใช้งานทั่วไปที่ไม่มีพื้นฐานด้านการเขียนโปรแกรม

ดังนั้นแนวทางการพัฒนาต่อไปคือการสร้าง Interface สำหรับตั้งค่าช่วงของเกจผ่าน Telegram โดยให้ผู้ใช้สามารถกำหนดค่าช่วงสีเขียว สีเหลือง และสีแดงได้เอง เช่น กำหนดว่าช่วงสีเขียวอยู่ระหว่าง 0–6 bar ช่วงสีเหลืองอยู่ระหว่าง 6–8 bar และช่วงสีแดงมากกว่า 8 bar จากนั้น ESP32S3-CAM จะบันทึกค่าที่ผู้ใช้ตั้งไว้ และนำค่าเหล่านี้ไปใช้ในการวิเคราะห์สถานะของเกจโดยอัตโนมัติ

แนวทางนี้จะทำให้ระบบมีความยืดหยุ่นมากขึ้น สามารถใช้งานกับเกจได้หลายประเภทโดยไม่ต้องแก้ไขโค้ดใหม่ทุกครั้ง ผู้ใช้สามารถปรับค่าช่วงการแจ้งเตือนได้เองผ่าน Telegram ทำให้ระบบเหมาะกับการใช้งานจริงมากขึ้น ทั้งในงานทดลอง งานซ่อมบำรุง และงานตรวจสอบเบื้องต้นในระบบอุตสาหกรรม

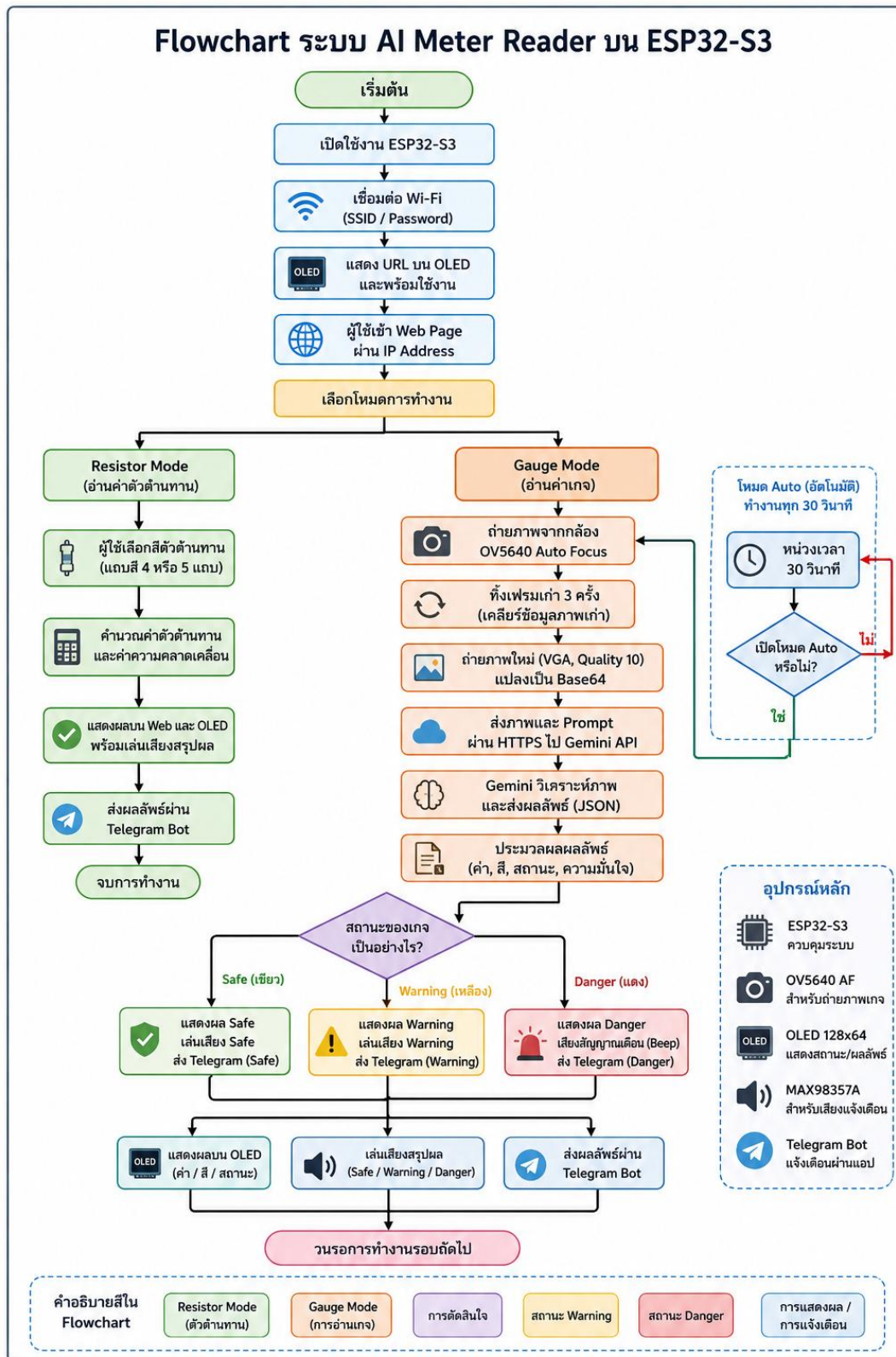
13.3 สรุปแนวทางการพัฒนา

โดยสรุป แนวทางการพัฒนาโครงการในอนาคตจะเน้นไปที่การเพิ่มความสะดวกในการใช้งานและลดความจำเป็นในการแก้ไขโค้ดโดยตรง โดยพัฒนา Interface การเลือกสีตัวต้านทานให้เป็นรูปแบบปุ่มสีที่เข้าใจง่าย และพัฒนา Interface สำหรับตั้งค่าช่วงสีของเกจผ่าน Telegram เพื่อให้ผู้ใช้สามารถปรับค่าการแจ้งเตือนได้เอง ระบบจึงมีความยืดหยุ่น ใช้งานง่าย และเหมาะสมกับการนำไปประยุกต์ใช้กับอุปกรณ์จริงมากยิ่งขึ้น

Link Youtube สำหรับดูการเดโมผลงาน :

<https://www.youtube.com/watch?v=fDfO5gYfs7E&feature=youtu.be>

14. ภาพผนวก Flow chart & Source code



Source code

```
#include "esp_camera.h"

#include <WiFi.h>

#include <WebServer.h>

#include <HTTPClient.h>

#include <WiFiClientSecure.h>

#include <ArduinoJson.h>

#include "base64.h"

#include <Wire.h>

#include <Adafruit_GFX.h>

#include <Adafruit_SSD1306.h>

#include "driver/i2s.h"


#include "Start.h"

#include "Resistormode.h"

#include "Gaugemode.h"

#include "Resistorcomplete.h"

#include "GaugeSafe.h"

#include "GaugeHigh.h"

#include "GaugeDanger.h"


// ===== USER CONFIG =====

const char* [redacted]

const char* [redacted]


String GEMINI_API_KEY = [redacted]

String TELEGRAM_BOT_TOKEN = [redacted]
```

String TELEGRAM_CHAT_ID =

// ===== OLED =====

#define OLED_SDA 1

#define OLED_SCL 2

#define SCREEN_WIDTH 128

#define SCREEN_HEIGHT 64

Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, -1);

// ===== SPEAKER =====

#define I2S_DOUT 41

#define I2S_LRC 40

#define I2S_BCLK 39

// เพิ่ม/ลดความดังเสียง WAV และเสียงเตือน

// 1.0 = เท่าเดิม, 2.0 = ดั้งขึ้นประมาณ 2 เท่า

// ถ้าเสียงแตกให้ลดลง เช่น 1.8 หรือ 2.0

#define WAV_VOLUME_GAIN 2.5

#define BEEP_VOLUME 30000

// ===== WEB / STATE =====

WebServer server(80);

String deviceURL = "";

String gaugeResult = "Not analyzed yet";

bool gaugeAutoMode = false;

bool dangerAlarmActive = false;

unsigned long lastGaugeAnalyze = 0;

```
unsigned long lastDangerBeep = 0;
```

```
const unsigned long GAUGE_INTERVAL = 30000;
```

```
// ===== CAMERA PINS =====
```

```
#define PWDN_GPIO_NUM    -1
```

```
#define RESET_GPIO_NUM  -1
```

```
#define XCLK_GPIO_NUM    15
```

```
#define SIOD_GPIO_NUM    4
```

```
#define SIOC_GPIO_NUM    5
```

```
#define Y9_GPIO_NUM      16
```

```
#define Y8_GPIO_NUM      17
```

```
#define Y7_GPIO_NUM      18
```

```
#define Y6_GPIO_NUM      12
```

```
#define Y5_GPIO_NUM      10
```

```
#define Y4_GPIO_NUM      8
```

```
#define Y3_GPIO_NUM      9
```

```
#define Y2_GPIO_NUM      11
```

```
#define VSYNC_GPIO_NUM   6
```

```
#define HREF_GPIO_NUM    7
```

```
#define PCLK_GPIO_NUM    13
```

```
// ===== SPEAKER =====
```

```
void setupSpeaker() {
```

```
    i2s_config_t config = {
```

```
        .mode = (i2s_mode_t)(I2S_MODE_MASTER | I2S_MODE_TX),
```

```
        .sample_rate = 24000,
```

```
        .bits_per_sample = I2S_BITS_PER_SAMPLE_16BIT,
```

```
        .channel_format = I2S_CHANNEL_FMT_ONLY_LEFT,
```

```

.communication_format = I2S_COMM_FORMAT_STAND_I2S,

.intr_alloc_flags = 0,

.dma_buf_count = 8,

.dma_buf_len = 512,

.use_apll = false,

.tx_desc_auto_clear = true,

.fixed_mclk = 0

};

```

```

i2s_pin_config_t pins = {

.bck_io_num = I2S_BCLK,

.ws_io_num = I2S_LRC,

.data_out_num = I2S_DOUT,

.data_in_num = I2S_PIN_NO_CHANGE

};

```

```

i2s_driver_install(I2S_NUM_0, &config, 0, NULL);

i2s_set_pin(I2S_NUM_0, &pins);

i2s_zero_dma_buffer(I2S_NUM_0);

}

```

```

int16_t amplifySample(int16_t sample, float gain) {

    int32_t amplified = (int32_t)(sample * gain);

    if (amplified > 32767) amplified = 32767;

    if (amplified < -32768) amplified = -32768;

    return (int16_t)amplified;
}

```

```
}
```

```
void playWav(const uint8_t *data, size_t len) {
```

```
// WAV ที่ใช้เป็น 24 kHz 16-bit mono และข้าม header 44 bytes
```

```
if (len <= 44) return;
```

```
const uint8_t *audio = data + 44;
```

```
size_t audioLen = len - 44;
```

```
// ส่งเป็น block เล็ก ๆ เพื่อปรับ volume ก่อนส่งเข้า MAX98357A
```

```
const int samplesPerBlock = 256;
```

```
int16_t buffer[samplesPerBlock];
```

```
size_t offset = 0;
```

```
while (offset + 1 < audioLen) {
```

```
    int count = 0;
```

```
    while (count < samplesPerBlock && offset + 1 < audioLen) {
```

```
        int16_t sample = (int16_t)(audio[offset] | (audio[offset + 1] << 8));
```

```
        buffer[count] = amplifySample(sample, WAV_VOLUME_GAIN);
```

```
        count++;
```

```
        offset += 2;
```

```
    }
```

```
    size_t written = 0;
```

```
    i2s_write(I2S_NUM_0, buffer, count * sizeof(int16_t), &written, portMAX_DELAY);
```

```
}
```

```
}
```

```
void playBeep(int freq, int ms, int volume) {
```

```
    int sampleRate = 24000;
```

```
    int samples = sampleRate * ms / 1000;
```

```
    for (int i = 0; i < samples; i++) {
```

```
        float t = (float)i / sampleRate;
```

```
        int16_t sample = sin(2.0 * PI * freq * t) * volume;
```

```
        size_t written;
```

```
        i2s_write(I2S_NUM_0, &sample, sizeof(sample), &written, portMAX_DELAY);
```

```
    }
```

```
}
```

```
void dangerBeepLoop() {
```

```
    if (!dangerAlarmActive) return;
```

```
    if (millis() - lastDangerBeep >= 900) {
```

```
        lastDangerBeep = millis();
```

```
        playBeep(2200, 250, BEEP_VOLUME);
```

```
        delay(80);
```

```
        playBeep(1800, 250, BEEP_VOLUME);
```

```
    }
```

```
}
```

```
// ===== OLED =====
```

```
void oledShow(String a, String b, String c = "", String d = "") {
```

```
    display.clearDisplay();
```

```

display.setTextColor(SSD1306_WHITE);

display.setTextSize(1);

display.setCursor(0, 0);

display.println("AI Meter Reader");

display.drawLine(0, 10, 127, 10, SSD1306_WHITE);

display.setCursor(0, 16); display.println(a);

display.setCursor(0, 29); display.println(b);

display.setCursor(0, 42); display.println(c);

display.setCursor(0, 55); display.println(d);

display.display();

}

```

```
// ===== UI =====
```

```

String css() {

    return R"rawliteral(

<style>

body{margin:0;font-family:Arial;background:#0f172a;color:white;text-align:center;}

h1{font-size:28px;margin-top:24px;}

.card{background:#1e293b;margin:16px;padding:18px;border-radius:22px;white-space:pre-line;}

select,button{width:88%;height:55px;margin:8px;border:0;border-radius:15px;font-size:18px;}

button{font-weight:bold;}

.green{background:#22c55e;}

.orange{background:#f97316;}

.yellow{background:#eab308;}

.blue{background:#3b82f6;color:white;}

.gray{background:#64748b;color:white;}

.red{background:#ef4444;color:white;}

img{width:94%;border-radius:18px;background:#020617;}

```

```
</style>
```

```
)rawliteral";
```

```
}
```

```
String homePage() {
```

```
    return "<html><head><meta name='viewport' content='width=device-width, initial-scale=1'>" + css() +
```

```
    "</head><body><h1>AI Meter Reader</h1>"
```

```
    "<div class='card'>Choose Mode</div>"
```

```
    "<button class='green' onclick=\"location.href='/resistor'\">Resistor Calculator</button>"
```

```
    "<button class='orange' onclick=\"location.href='/gauge'\">Gauge Analyzer</button>"
```

```
    "</body></html>";
```

```
}
```

```
// ===== RESISTOR =====
```

```
String digitOptions() {
```

```
    return "<option value='0'>Black 0</option><option value='1'>Brown 1</option><option value='2'>Red 2</option><option value='3'>Orange  
3</option><option value='4'>Yellow 4</option><option value='5'>Green 5</option><option value='6'>Blue 6</option><option value='7'>Violet  
7</option><option value='8'>Gray 8</option><option value='9'>White 9</option>";
```

```
}
```

```
String multiplierOptions() {
```

```
    return "<option value='1'>Black x1</option><option value='10'>Brown x10</option><option value='100'>Red x100</option><option  
value='1000'>Orange x1k</option><option value='10000'>Yellow x10k</option><option value='100000'>Green x100k</option><option  
value='1000000'>Blue x1M</option><option value='10000000'>Violet x10M</option><option value='0.1'>Gold x0.1</option><option value='0.01'>Silver  
x0.01</option>";
```

```
}
```

```
String toleranceOptions() {
```

```

    return "<option value='1'>Brown ±1%</option><option value='2'>Red ±2%</option><option value='0.5'>Green ±0.5%</option><option
value='0.25'>Blue ±0.25%</option><option value='0.1'>Violet ±0.1%</option><option value='5'>Gold ±5%</option><option value='10'>Silver
±10%</option>";
}

```

```

String resistorPage() {

    return "<html><head><meta name='viewport' content='width=device-width, initial-scale=1'>" + css() +

    "</head><body><h1>Resistor Calculator</h1>"

    "<div class='card'><h3>4 Band</h3><form action='/calc4'>"

    "<select name='b1'>" + digitOptions() + "</select>"

    "<select name='b2'>" + digitOptions() + "</select>"

    "<select name='mul'>" + multiplierOptions() + "</select>"

    "<select name='tol'>" + toleranceOptions() + "</select>"

    "<button class='yellow' type='submit'>Calculate 4 Band</button></form></div>"

    "<div class='card'><h3>5 Band</h3><form action='/calc5'>"

    "<select name='b1'>" + digitOptions() + "</select>"

    "<select name='b2'>" + digitOptions() + "</select>"

    "<select name='b3'>" + digitOptions() + "</select>"

    "<select name='mul'>" + multiplierOptions() + "</select>"

    "<select name='tol'>" + toleranceOptions() + "</select>"

    "<button class='yellow' type='submit'>Calculate 5 Band</button></form></div>"

    "<button class='gray' onclick=\"location.href='/\">Back Home</button>"

    "</body></html>";
}

```

```

String formatOhm(float value) {

    if (value >= 1000000) return String(value / 1000000.0, 2) + " M ohm";

    if (value >= 1000) return String(value / 1000.0, 2) + " k ohm";

    return String(value, 2) + " ohm";
}

```

```
}
```

```
void showResistorResult(String result) {
```

```
    oledShow("Mode: RESISTOR", "Result:", result, "Offline calc");
```

```
    String html = "<html><head><meta name='viewport' content='width=device-width, initial-scale=1'> + css() +
```

```
    "</head><body><h1>Result</h1>"
```

```
    "<div class='card'><h2>" + result + "</h2><p>Calculated by ESP32-S3 offline</p></div>"
```

```
    "<button class='green' onclick=\"location.href='/resistor'\">Calculate Again</button>"
```

```
    "<button class='gray' onclick=\"location.href='/'\">Back Home</button>"
```

```
    "</body></html>";
```

```
    server.send(200, "text/html", html);
```

```
    playWav(Resistorcomplete, sizeof(Resistorcomplete));
```

```
}
```

```
void handleCalc4() {
```

```
    int b1 = server.arg("b1").toInt();
```

```
    int b2 = server.arg("b2").toInt();
```

```
    float mul = server.arg("mul").toFloat();
```

```
    String tol = server.arg("tol");
```

```
    float value = ((b1 * 10) + b2) * mul;
```

```
    showResistorResult(formatOhm(value) + " ±" + tol + "%");
```

```
}
```

```
void handleCalc5() {
```

```
    int b1 = server.arg("b1").toInt();
```

```
    int b2 = server.arg("b2").toInt();
```

```

int b3 = server.arg("b3").toInt();

float mul = server.arg("mul").toFloat();

String tol = server.arg("tol");

float value = ((b1 * 100) + (b2 * 10) + b3) * mul;

showResistorResult(formatOhm(value) + " ±" + tol + "%");

}

// ===== GAUGE UI =====

String gaugePage() {

    String autoText = gaugeAutoMode ? "AUTO MONITOR: ON / 30 sec" : "AUTO MONITOR: OFF";

    return "<html><head><meta name='viewport' content='width=device-width, initial-scale=1'>" + css() +

    "</head><body><h1>Gauge Analyzer</h1>"

    "<div class='card'><b>" + autoText + "</b><br><br><b>Last Result:</b><br>" + gaugeResult + "</div>"

    "<button class='blue' onclick=\"location.href='/gauge_preview'>Preview Once</button>"

    "<button class='yellow' onclick=\"location.href='/gauge_analyze'>Analyze Now</button>"

    "<button class='green' onclick=\"location.href='/gauge_start'>Start Auto 30s</button>"

    "<button class='red' onclick=\"location.href='/gauge_stop'>Stop Auto</button>"

    "<button class='gray' onclick=\"location.href='/'>Back Home</button>"

    "</body></html>";

}

void handleGaugePreview() {

    String html = "<html><head><meta name='viewport' content='width=device-width, initial-scale=1'>" + css() +

    "</head><body><h1>Gauge Preview</h1>"

    "<div class='card'><img src='/jpg?t=" + String(millis()) + "'></div>"

    "<button class='yellow' onclick=\"location.href='/gauge_analyze'>Analyze This</button>"

    "<button class='gray' onclick=\"location.href='/gauge'>Back Gauge</button>"

```

```
"</body></html>";
```

```
oledShow("Mode: GAUGE", "Preview once", "VGA q10", "");
```

```
server.send(200, "text/html", html);
```

```
}
```

```
// ===== TELEGRAM =====
```

```
void sendTelegram(String msg) {
```

```
    WiFiClientSecure client;
```

```
    client.setInsecure();
```

```
    HTTPClient https;
```

```
    String url = "https://api.telegram.org/bot" + TELEGRAM_BOT_TOKEN + "/sendMessage";
```

```
    https.begin(client, url);
```

```
    https.addHeader("Content-Type", "application/x-www-form-urlencoded");
```

```
    String body = "chat_id=" + TELEGRAM_CHAT_ID + "&text=" + msg;
```

```
    int code = https.POST(body);
```

```
    String response = https.getString();
```

```
    Serial.print("Telegram code: ");
```

```
    Serial.println(code);
```

```
    Serial.println(response);
```

```
    https.end();
```

```
}
```

```
// ===== GEMINI =====
```

```
String askGeminiGauge(camera_fb_t *fb) {
```

```
    String img64 = base64::encode(fb->buf, fb->len);
```

```
    String prompt =
```

```
"You are a professional industrial gauge inspector. "
```

```
"The gauge in the image is a HYDRO-PNEUMATIC PRESSURE GAUGE with a fixed scale from 0 to 600 bar. "
```

```
"IMPORTANT RULES: "
```

```
"1. Use ONLY the visible image. "
```

```
"2. Read the NEEDLE TIP, not the needle base. "
```

```
"3. Assume the gauge scale is 0 to 600 bar. "
```

```
"4. Major numbers are 0, 50, 100, 150, 200, 250, 300, 350, 400, 450, 500, 550, 600. "
```

```
"5. Estimate intermediate values proportionally between tick marks. "
```

```
"6. Return the nearest 10 bar. "
```

```
"7. If the needle is between two values, interpolate from the angle. "
```

```
"8. If uncertain, choose the closest visible value. "
```

```
"9. Never answer UNKNOWN unless the needle is not visible. "
```

```
"COLOR ZONES: "
```

```
"GREEN = 0 to 300 bar. "
```

```
"YELLOW = above 300 to below 500 bar. "
```

```
"RED = 500 bar and above. "
```

```
"SAFETY RULE: "
```

```
"If the needle is very close to a color boundary, choose the higher-risk zone. "
```

"ANALYSIS STEPS: "

"Step 1: Locate the center pivot. "

"Step 2: Locate the needle tip. "

"Step 3: Determine the needle angle. "

"Step 4: Convert the angle to a value on the 0-600 bar scale. "

"Step 5: Determine the color zone where the needle tip is located. "

"Step 6: Determine status. "

"STATUS RULES: "

"GREEN = SAFE. "

"YELLOW = WARNING. "

"RED = DANGER. "

"Reply ONLY in this exact format: \n"

"VALUE: <number> bar\n"

"COLOR: GREEN or YELLOW or RED\n"

"STATUS: SAFE or WARNING or DANGER\n"

"CONFIDENCE: HIGH or MEDIUM or LOW\n"

"DETAIL: <short explanation>;

String payload =

```
"{"contents":{"parts":["
```

```
"{"text":"","prompt":"
```

```
"{"inline_data":{"mime_type":"image/jpeg","data":"","img64":"
```

```
"]}"]
```

Serial.print("Payload size: ");

```
Serial.println(payload.length());
```

```
WiFiClientSecure client;
```

```
client.setInsecure();
```

```
HTTPClient https;
```

```
https.setTimeout(30000);
```

```
String url = "https://generativelanguage.googleapis.com/v1beta/models/gemini-2.5-flash:generateContent?key=" + GEMINI_API_KEY;
```

```
https.begin(client, url);
```

```
https.addHeader("Content-Type", "application/json");
```

```
int code = https.POST(payload);
```

```
String response = https.getString();
```

```
Serial.print("Gemini HTTP code: ");
```

```
Serial.println(code);
```

```
Serial.println(response);
```

```
https.end();
```

```
if (code <= 0) return "VALUE: unknown\nCOLOR: UNKNOWN\nSTATUS: ERROR\nREASON: Gemini request failed";
```

```
if (code != 200) return "VALUE: unknown\nCOLOR: UNKNOWN\nSTATUS: ERROR\nREASON: Gemini HTTP error " + String(code);
```

```
DynamicJsonDocument doc(30000);
```

```
DeserializationError err = deserializeJson(doc, response);
```

```
if (err) return "VALUE: unknown\nCOLOR: UNKNOWN\nSTATUS: ERROR\nREASON: Gemini JSON parse failed";
```

```

const char* text = doc["candidates"][0]["content"][0]["text"];

if (!text) return "VALUE: unknown\nCOLOR: UNKNOWN\nSTATUS: ERROR\nREASON: No Gemini text result";

return String(text);
}

```

```

String getLineValue(String src, String key) {

    int start = src.indexOf(key);

    if (start < 0) return "";

    start += key.length();

    int end = src.indexOf("\n", start);

    if (end < 0) end = src.length();

    String out = src.substring(start, end);

    out.trim();

    return out;
}

```

```

void playGaugeSoundByResult(String result) {

    String status = getLineValue(result, "STATUS:");

    String color = getLineValue(result, "COLOR:");

    if (status.indexOf("DANGER") >= 0 || color.indexOf("RED") >= 0) {

        playWav(GaugeDanger, sizeof(GaugeDanger));

        dangerAlarmActive = true;

        lastDangerBeep = 0;

    }

    else if (status.indexOf("WARNING") >= 0 || color.indexOf("YELLOW") >= 0) {

```

```

    dangerAlarmActive = false;

    playWav(GaugeHigh, sizeof(GaugeHigh));

}

else if (status.indexOf("SAFE") >= 0 || color.indexOf("GREEN") >= 0) {

    dangerAlarmActive = false;

    playWav(GaugeSafe, sizeof(GaugeSafe));

}

}

void sendTelegramByStatus(String result) {

    String value = getLineValue(result, "VALUE:");

    String color = getLineValue(result, "COLOR:");

    String status = getLineValue(result, "STATUS:");

    String msg;

    if (status.indexOf("DANGER") >= 0 || color.indexOf("RED") >= 0) {

        msg = "  DANGER\n";

    } else if (status.indexOf("WARNING") >= 0 || color.indexOf("YELLOW") >= 0) {

        msg = "  WARNING / HIGH\n";

    } else if (status.indexOf("SAFE") >= 0 || color.indexOf("GREEN") >= 0) {

        msg = "  SAFE\n";

    } else {

        msg = "  GAUGE RESULT\n";

    }

    msg += "\nValue: " + value;

```

```
msg += "\nColor: " + color;

msg += "\nStatus: " + status;

msg += "\n\nRaw:\n" + result;

msg += "\n\nTime: " + String(millis() / 1000) + " sec";
```

```
sendTelegram(msg);
```

```
}
```

```
void analyzeGaugeOnce(bool fromAuto) {
```

```
oledShow(fromAuto ? "AUTO GAUGE" : "Mode: GAUGE", "Capturing...", "Gemini analyzing", "");
```

```
// ทิ้งเฟรมเก่าก่อน เพื่อกัน Gemini อ่านภาพค้างจากรอบก่อน
```

```
for (int i = 0; i < 3; i++) {
```

```
camera_fb_t * oldFb = esp_camera_fb_get();
```

```
if (oldFb) esp_camera_fb_return(oldFb);
```

```
delay(200);
```

```
}
```

```
// ถ่ายภาพใหม่จริงสำหรับส่งไปวิเคราะห์
```

```
camera_fb_t * fb = esp_camera_fb_get();
```

```
if (!fb) {
```

```
gaugeResult = "VALUE: unknown\nCOLOR: UNKNOWN\nSTATUS: ERROR\nREASON: Capture failed";
```

```
oledShow("GAUGE", "Capture failed", "", "");
```

```
sendTelegramByStatus(gaugeResult);
```

```
return;
```

```
}
```

```
Serial.print("Image size: ");
```

```
Serial.println(fb->len);
```

```
gaugeResult = askGeminiGauge(fb);
```

```
esp_camera_fb_return(fb);
```

```
String value = getLineValue(gaugeResult, "VALUE:");
```

```
String color = getLineValue(gaugeResult, "COLOR:");
```

```
String status = getLineValue(gaugeResult, "STATUS:");
```

```
oledShow("GAUGE result", value, color, status);
```

```
sendTelegramByStatus(gaugeResult);
```

```
playGaugeSoundByResult(gaugeResult);
```

```
}
```

```
void handleGaugeAnalyze() {
```

```
    analyzeGaugeOnce(false);
```

```
    server.send(200, "text/html", gaugePage());
```

```
}
```

```
void handleGaugeStart() {
```

```
    dangerAlarmActive = false;
```

```
    gaugeAutoMode = true;
```

```
    lastGaugeAnalyze = 0;
```

```
    gaugeResult = "Auto monitor started. Analyze every 30 sec.";
```

```
    oledShow("Mode: GAUGE", "AUTO ON", "Every 30 sec", "VGA q10");
```

```
    server.send(200, "text/html", gaugePage());
```

```
}
```

```
void handleGaugeStop() {
```

```
    dangerAlarmActive = false;
```

```
    gaugeAutoMode = false;
```

```
    gaugeResult = "Auto monitor stopped.";
```

```
    oledShow("Mode: GAUGE", "AUTO OFF", "Stopped", "");
```

```
    server.send(200, "text/html", gaugePage());
```

```
}
```

```
// ===== JPG SNAPSHOT =====
```

```
void handleJPG() {
```

```
    camera_fb_t * fb = esp_camera_fb_get();
```

```
    if (!fb) {
```

```
        server.send(500, "text/plain", "Capture failed");
```

```
        return;
```

```
    }
```

```
WiFiClient client = server.client();
```

```
client.println("HTTP/1.1 200 OK");
```

```
client.println("Content-Type: image/jpeg");
```

```
client.print("Content-Length: ");
```

```
client.println(fb->len);
```

```
client.println("Connection: close");
```

```
client.println();
```

```
client.write(fb->buf, fb->len);

esp_camera_fb_return(fb);

}


// ===== CAMERA SETUP =====

void setupCamera() {

    camera_config_t config;

    config.ledc_channel = LEDC_CHANNEL_0;

    config.ledc_timer = LEDC_TIMER_0;


    config.pin_d0 = Y2_GPIO_NUM;

    config.pin_d1 = Y3_GPIO_NUM;

    config.pin_d2 = Y4_GPIO_NUM;

    config.pin_d3 = Y5_GPIO_NUM;

    config.pin_d4 = Y6_GPIO_NUM;

    config.pin_d5 = Y7_GPIO_NUM;

    config.pin_d6 = Y8_GPIO_NUM;

    config.pin_d7 = Y9_GPIO_NUM;


    config.pin_xclk = XCLK_GPIO_NUM;

    config.pin_pclk = PCLK_GPIO_NUM;

    config.pin_vsync = VSYNC_GPIO_NUM;

    config.pin_href = HREF_GPIO_NUM;


    config.pin_sccb_sda = SIOD_GPIO_NUM;

    config.pin_sccb_scl = SIOC_GPIO_NUM;
```

```
config.pin_pwdn = PWDN_GPIO_NUM;
```

```
config.pin_reset = RESET_GPIO_NUM;
```

```
config.xclk_freq_hz = 20000000;
```

```
config.pixel_format = PIXFORMAT_JPEG;
```

```
config.frame_size = FRAMESIZE_VGA;
```

```
config.jpeg_quality = 10;
```

```
config.fb_count = 2;
```

```
config.fb_location = CAMERA_FB_IN_PSRAM;
```

```
config.grab_mode = CAMERA_GRAB_LATEST;
```

```
esp_err_t err = esp_camera_init(&config);
```

```
if (err != ESP_OK) {
```

```
    Serial.printf("Camera failed: 0x%x\n", err);
```

```
    oledShow("Camera failed", String(err, HEX), "", "");
```

```
    while (true) delay(1000);
```

```
}
```

```
sensor_t * s = esp_camera_sensor_get();
```

```
Serial.print("Sensor PID: 0x");
```

```
Serial.println(s->id.PID, HEX);
```

```
s->set_framesize(s, FRAMESIZE_VGA);
```

```
s->set_quality(s, 10);
```

```
s->set_vflip(s, 0);
```

```
s->set_hmirror(s, 0);
```

```
Serial.println("Camera init OK VGA q10");
```

```
}
```

```
// ===== HANDLERS =====
```

```
void handleHome() {
```

```
    dangerAlarmActive = false;
```

```
    gaugeAutoMode = false;
```

```
    oledShow("Phone connected", "Mode: HOME", WiFi.localIP().toString(), "");
```

```
    server.send(200, "text/html", homePage());
```

```
}
```

```
void handleResistor() {
```

```
    dangerAlarmActive = false;
```

```
    gaugeAutoMode = false;
```

```
    oledShow("Mode: RESISTOR", "Offline calculate", "4/5 Band", "");
```

```
    server.send(200, "text/html", resistorPage());
```

```
    playWav(Resistormode, sizeof(Resistormode));
```

```
}
```

```
void handleGauge() {
```

```
    dangerAlarmActive = false;
```

```
    oledShow("Mode: GAUGE", gaugeAutoMode ? "AUTO ON" : "Manual mode", "VGA q10", "");
```

```
    server.send(200, "text/html", gaugePage());
```

```
    playWav(Gaugemode, sizeof(Gaugemode));
```

```
}
```

```
void testInternet() {

  WiFiClientSecure client;

  client.setInsecure();


  HTTPClient https;

  https.setTimeout(10000);

  https.begin(client, "https://www.google.com");

  int code = https.GET();


  Serial.print("Internet test code: ");

  Serial.println(code);

  https.end();

}


// ===== SETUP =====

void setup() {

  Serial.begin(115200);

  delay(1000);


  setupSpeaker();


  Wire.begin(OLED_SDA, OLED_SCL);

  display.begin(SSD1306_SWITCHCAPVCC, 0x3C);


  oledShow("Booting...", "Camera init", "VGA q10", "");

  setupCamera();


  oledShow("Connecting WIFI", ssid, "", "");
```

```
WiFi.mode(WIFI_STA);

WiFi.begin(ssid, password);

while (WiFi.status() != WL_CONNECTED) {

    delay(500);

    Serial.print(".");

}

deviceURL = "http://" + WiFi.localIP().toString();

Serial.println();

Serial.print("Open: ");

Serial.println(deviceURL);

testInternet();

oledShow("WiFi connected", deviceURL, "Open on phone", "");

playWav(Start, sizeof(Start));

sendTelegram("🟢 ESP32 AI Meter Reader Online\nOpen: " + deviceURL);

server.on("/", handleHome);

server.on("/resistor", handleResistor);

server.on("/calc4", handleCalc4);

server.on("/calc5", handleCalc5);

server.on("/gauge", handleGauge);

server.on("/gauge_preview", handleGaugePreview);
```

```

server.on("/gauge_analyze", handleGaugeAnalyze);

server.on("/gauge_start", handleGaugeStart);

server.on("/gauge_stop", handleGaugeStop);

server.on("/jpg", handleJPG);


server.begin();

Serial.println("Web server started");

}


// ===== LOOP =====

void loop() {

  server.handleClient();

  dangerBeepLoop();

  if (gaugeAutoMode) {

    unsigned long now = millis();

    if (lastGaugeAnalyze == 0 || now - lastGaugeAnalyze >= GAUGE_INTERVAL) {

      lastGaugeAnalyze = now;

      analyzeGaugeOnce(true);

    }

  }

}

```